

Tối ưu hóa Pipe để Cải thiện Thông lượng IPC trong xv6

Mã dự án: 25236

Nguyễn Sỹ Tuấn Lê Huy Sơn Vũ Tiến Long

Trường Điện – Điện tử · Đại học Bách Khoa Hà Nội

Hệ điều hành [Multimedia] – 20252

Phân công công việc & Đóng góp

STT	Thành viên	Nội dung công việc	%
1	Nguyễn Sỹ Tuấn	v2 – Bulk Copy	15%
2		v4 – Multi-page Allocation	15%
4		Baseline v1, viết pipebench.c và báo cáo	4%
5	Lê Huy Sơn	v3 – Ring of Buffers	15%
6		v6 – Priority Boost	15%
7		v8 – All Combined (tích hợp v3, v6)	3%
8	Vũ Tiến Long	v5 – Lazy Wakeup	15%
9		v7 – Cache-line Alignment	15%
10		v8 – All Combined (tích hợp v5, v7)	3%
Tổng			100%

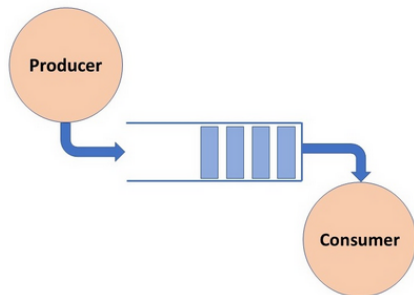
Giới thiệu

Các phiên bản tối ưu

Kết quả thực nghiệm

- ▶ Pipe là cơ chế IPC **unidirectional**, **half-duplex**
- ▶ Hiện thực dưới dạng **bounded buffer** trong kernel
- ▶ Hai đầu: `fd[0]` (đọc) và `fd[1]` (ghi)
- ▶ Dữ liệu hoàn toàn trên RAM, không qua đĩa

Pipe được minh họa theo cơ chế **Producer-Consumer Problem** (Dijkstra, 1965): tiến trình ghi đóng vai *Producer*, tiến trình đọc đóng vai *Consumer*, bộ đệm vòng là *Buffer* dùng chung.



$$\text{Throughput (B/s)} = \frac{\text{total_bytes}}{\text{time (s)}} = \frac{\text{total_bytes} \times \text{TICKS_PER_SEC}}{\text{elapsed_ticks}}$$

- ▶ **total_bytes**: tổng số byte cần truyền qua pipe
- ▶ **TICKS_PER_SEC**: số tick trong 1 giây (xv6: timer interrupt tăng tick, ≈ 10 tick/s)
- ▶ **elapsed_ticks**: thời gian truyền hết dữ liệu tính bằng tick (uptime() syscall)

Đơn vị đo: **B/s** (hoặc KB/s, MB/s). Trong báo cáo dùng $1 \text{ MB/s} = 2^{20} \text{ B/s}$.

Cấu trúc các trường trong struct pipe

```
#define PIPESIZE 512

struct pipe {
    struct spinlock lock;
    char data[PIPESIZE];
    uint nread;
    uint nwrite;
    int readopen;
    int writeopen;
};
```

- ▶ **lock** – spinlock bảo vệ toàn bộ struct, ngăn race condition
- ▶ **data[]** – bộ đệm vòng 512 byte
- ▶ **nread, nwrite** – chỉ số tích lũy (không reset):
 - ▷ Đầy: `nwrite == nread + PIPESIZE`
 - ▷ Rỗng: `nread == nwrite`
 - ▷ Truy cập: `data[nwrite % PIPESIZE]`
- ▶ **readopen, writeopen** – cờ báo đầu pipe còn mở

- ▶ **pipealloc()**: cấp 1 trang cho pipe và tạo 2 file descriptor `f0`, `f1` để đọc và ghi
- ▶ **pipewrite()**: ghi dữ liệu từ user space vào bộ đệm vòng; nếu buffer đầy thì sleep chờ reader
- ▶ **piperead()**: đọc dữ liệu từ bộ đệm vòng ra user space; nếu buffer rỗng thì sleep chờ writer
- ▶ **pipeclose()**: đóng một đầu pipe, đánh thức bên đối diện; giải phóng bộ nhớ khi cả hai đầu đã đóng

- ▶ **Chi phí gọi syscall:**
- ▶ **Kích thước pipe buffer:** $\text{PIPESIZE} = 512 \text{ B}$ quá nhỏ.
- ▶ **copyin/copyout:** copy từng byte một thay vì copy nhiều byte mỗi lần.
- ▶ **Spinlock & sleep/wakeup:** wakeup() vô điều kiện phải quét toàn bộ `proc[]` mỗi lần ghi.
- ▶ **Scheduler:** không ưu tiên reader/writer khi pipe có dữ liệu.
- ▶ **CPU cache – False sharing:** `nread` và `nwrite` cùng cache line khiến CPU invalidate cache liên tục trên SMP

- ▶ `chunk_bytes` càng lớn thì số lần gọi syscall càng ít, giúp giảm overhead trap vào kernel
- ▶ Số lần gọi syscall = $\text{total_bytes} / \text{chunk_bytes}$ – chunk nhỏ $\Rightarrow$ băng thông bị giới hạn bởi chi phí syscall
- ▶ Tuy nhiên chunk lớn hơn đồng nghĩa mỗi lần `write()` phải chờ reader drain buffer nhiều lần hơn, làm tăng chi phí `sleep()/wakeup()`
- ▶ Đây là sự **đánh đổi** giữa ít syscall và nhiều lần chờ đồng bộ
- ▶ Đo băng thông với `chunk_bytes = 64, 128, 256, 512, 1024, 2048, 4096` để thấy rõ sự thay đổi

Giải quyết vấn đề:

- ▶ v1 gọi `copyin()/copyout()` từng byte một $\Rightarrow n$ lần cho n byte

Cách giải quyết:

- ▶ Ghi/đọc cả khối `chunk_bytes` trong một lần `copyin()`
- ▶ Tính số byte có thể copy an toàn: `min(còn_lại, free_slots, till_wrap)`
- ▶ Số lần `copyin()` giảm từ n xuống $\lceil n/\text{chunk_bytes} \rceil$

Source code thay đổi:

```
int to_copy = min(n-i, free_slots, till_wrap);
copyin(pr->pagetable, &pi->data[head_slot], addr+i, to_copy);
pi->nwrite += to_copy;
```

Giải quyết vấn đề:

- ▶ v2 vẫn bị giới hạn bởi $\text{PIPESIZE} = 512 \text{ B}$; chunk lớn làm buffer đầy liên tục
- ▶ Wrap-around: ghi sát cuối mảng buộc phải gọi `copyin()` 2 lần

Cách giải quyết:

- ▶ Tổ chức buffer thành 2D array `data[N_BUFS][BUFSIZE]` ($4 \times 512 \text{ B} = 2048 \text{ B}$)
- ▶ Mỗi lần `copyin()` nằm gọn trong 1 sub-buffer, không vượt biên

Source code thay đổi:

```
char data[N_BUFS][BUFSIZE];    // thay vi char data[PIPESIZE]

uint buf_idx = (nwrite % PIPE_TOTAL) / BUFSIZE;
uint buf_off = (nwrite % PIPE_TOTAL) % BUFSIZE;
copyin(..., &pi->data[buf_idx][buf_off], addr+i, to_copy);
```

Giải quyết vấn đề:

- ▶ Metadata (spinlock, nread, nwrite) và data[] cùng trang → ghi data liên tục evict metadata khỏi cache → cache miss khi cập nhật nwrite

Cách giải quyết:

- ▶ Tách 2 trang riêng: trang 1 chứa struct (metadata), trang 2 chứa data buffer 4096 B
- ▶ Metadata luôn nằm trong cache, không bị evict bởi data writes

Source code thay đổi:

```
struct pipe { char *data; ... };    // con tro thay vi mang nhung

// pipealloc:
pi->data = (char*) kalloc();        // cap trang thu 2 cho data
```

Giải quyết vấn đề:

- ▶ v1–v4 gọi wakeup() vô điều kiện sau mỗi write/read → quét toàn bộ proc[64] kể cả khi reader không ngủ

Cách giải quyết:

- ▶ Writer chỉ wakeup(reader) nếu buffer rỗng lúc bắt đầu ghi (reader có thể đang ngủ)
- ▶ Reader chỉ wakeup(writer) nếu buffer đầy lúc bắt đầu đọc (writer có thể đang ngủ)

Source code thay đổi:

```
// pipewrite: chỉ wakeup khi cần
int need_wakeup = (pi->nwrite == pi->nread);
...
if(need_wakeup && i > 0) wakeup(&pi->nread);

// piperead: chỉ wakeup khi cần
int was_full = (pi->nwrite == pi->nread + PIPE_TOTAL);
```

Giải quyết vấn đề:

- ▶ Sau wakeup(), reader/writer chuyển sang RUNNABLE nhưng phải xếp hàng chờ scheduler round-robin → trễ không cần thiết

Cách giải quyết:

- ▶ Thêm trường priority vào struct proc
- ▶ wakeup() đặt priority = PRIORITY_BOOST (10) → scheduler chọn ngay
- ▶ yield() giảm priority mỗi tick để tránh starvation

Source code thay đổi:

```
// wakeup(): dat priority cao nhat khi danh thuc
p->state = RUNNABLE;
p->priority = PRIORITY_BOOST; // = 10

// yield(): giam priority moi tick
if(p->priority > PRIORITY_BASE) p->priority--;
```

Giải quyết vấn đề:

- ▶ `nwrite` và `nread` cùng cache line 64 B → false sharing trên multi-core: Core 0 ghi `nwrite` làm cache line của Core 1 invalid → stall

Cách giải quyết:

- ▶ Thêm padding để `nwrite` và `nread` nằm trên 2 cache line riêng biệt

Source code thay đổi:

```
struct pipe {  
    struct spinlock lock; char *data; uint nwrite;  
    char _pad_w[28];    // cache line 0 (byte 0-63): writer-side  
    uint nread;  
    char _pad_r[52];    // cache line 1 (byte 64-127): reader-side  
    int readopen; int writeopen;  
};
```

Giải quyết vấn đề:

- ▶ Tổng hợp tất cả 5 cơ chế tối ưu của v2–v7 vào một phiên bản duy nhất

Cách giải quyết: không thêm kỹ thuật mới, chỉ kết hợp:

Cơ chế	Nguồn	Bottleneck giải quyết
Bulk copyin/copyout	v2	n page walks $\rightarrow \leq 2/\text{write}$
Trang data riêng (4 KB)	v4	metadata và data tách biệt
Lazy wakeup	v5	quét proc[] vô điều kiện
Priority boost	v6	scheduler delay sau wakeup
Cache-line alignment	v7	false sharing trên multi-core

Thông lượng theo chunk size – CPUS=1 (MB/s)

chunk	v1	v2	v3	v4	v5	v6	v7	v8
64 B	0.19	0.26	0.27	0.27	0.95	1.00	0.98	0.98
128 B	0.30	0.48	0.53	0.55	1.82	1.91	1.82	1.82
256 B	0.40	0.82	1.00	1.05	3.08	3.33	3.33	3.33
512 B	0.49	1.33	1.91	2.00	5.00	5.00	5.72	5.00
1024 B	0.49	1.54	3.08	3.64	6.67	8.00	8.00	8.00
2048 B	0.53	1.67	5.00	5.72	8.00	10.00	10.00	10.00
4096 B	0.53	1.74	5.72	10.00	8.00	10.00	10.00	13.33

Nhận xét

v5@4096B giảm so với v4 (lazy wakeup overhead khi buffer liên tục đầy). v8 đạt 13.33 MB/s – tăng 25× so với v1.

chunk	v7		v8	
	CPUS=1	CPUS=2	CPUS=1	CPUS=2
64 B	0.98	1.21	0.98	1.29
128 B	1.82	1.91	1.82	2.50
256 B	3.33	3.33	3.33	3.33
512 B	5.72	5.00	5.00	5.72
1024 B	8.00	8.00	8.00	6.67
2048 B	10.00	10.00	10.00	13.33
4096 B	10.00	20.00	10.00	13.33